Retail Sales Analytics Using Excel, SQL, Power BI and Python

Skills take away From This Project:

- Data Cleaning and Preprocessing using Excel
- Data Storage, Querying, and Transformation using SQL
- Data Analysis and Visualization using Python (Pandas, NumPy, Matplotlib/Seaborn)
- Customer Segmentation using Machine Learning (KMeans/Clustering)
- Business Intelligence and Reporting using Power BI
- End-to-End Data Pipeline Integration

Problem Statement:

In a competitive retail market, businesses need to track sales performance, understand customer behaviour, and identify growth opportunities. This project aims to build an end-to-end analytics solution that enables insights generation from raw transactional data using a multi-tool approach.

Approach:

Phase 1: Excel – Data Cleaning & Preparation:

- **Standardize Column Headers** - Ensured all columns across sheets (like customers, orders, products) are consistently named (e.g., use snake_case or camelCase).
- **Remove Duplicates** - Identifying and removing any duplicate records (e.g., in customers or order_items)
- **Handle Missing Values** - Use filtering and formulas to find blanks/nulls, Fill missing ZIP codes or phone numbers with placeholders or "NA".
- **Data Type Conversion -** Convert order dates, shipped dates, and required dates into Excel Date format, ensure numeric fields (quantity, price) are set to number format.
- **Data Validation** - Use dropdowns for fields like order_status (Pending, Shipped, Delivered) to ensure consistency.
- **Create New Derived Columns** - Add a column in order_items to calculate total_price = (list_price *quantity) - discount.
- **Merge Lookup Data -** Use VLOOKUP or XLOOKUP to merge product names into order_items using product_id.
- **Basic Pivot Table** - Build a pivot table summarizing total sales by product category.
- **Sort and Filter for Outliers** - Find outliers in pricing (e.g., products with very high or low list_price).
- **Preparing Final CSVs** - Save cleaned sheets as CSVs named: customers.csv, orders.csv, etc for SQL import.

Phase 2: SQL – Database Management and Querying:

- **Creating Tables Based on ERD** - Use CREATE TABLE statements to replicate the exact structure of the ER diagram (with constraints).
- **Importing CSVs into SQL** - Load cleaned Excel files using LOAD DATA or MySQL Workbench import feature.
- **Inner Join for Order Details** - Join orders, order_items, and products to display detailed line items.
- **Total Sales by Store** - Write a query to group sales (total_price) by each store_id.
- **Top 5 Selling Products** - Use ORDER BY and LIMIT to get the top 5 most sold products by quantity.
- **Customer Purchase Summary** - For each customer, return total orders placed, total items purchased, and total revenue.
- **Segment Customers by Total Spend** - Write a query to classify customers into spending brackets (e.g., low, medium, high).
- **Staff Performance Analysis** - Analysed total revenue generated by each staff member based on their handled orders.
- **Stock Alert Query** - listing products where stock quantity < 10 in any store.
- **Final Segmentation Table** - A table customer_segments that will be populated from Python ML results later.

Phase 3: Python and ML Tasks:

- **Load Data from SQL** - Use pandas.read_sql() to pull the orders, order_items, and customers tables into a Data Frame.
- **Basic EDA** - Use df.describe(), df.info(), and df.value_counts() to summarize the dataset, Plot basic charts using matplotlib or seaborn.
- **Calculate RFM Features for Customers:**
    1. Compute Recency, Frequency, and Monetary values for each customer.
    2. Recency: Days since last order
    3. Frequency: Number of orders
    4. Monetary: Total value of all purchases
- **Export Segmentation Results to SQL** - Save the rfm_data with the segment label to SQL as customer_segments table.

Phase 4: Power BI – Visualization & Dashboarding:

- **Connect Power BI to SQL** - Import key tables like orders, products, order_items, customers, stores, and staffs from the SQL database.
- **Create Relationships Between Tables -** Use Model View to define foreign key relationships based on the ER diagram (e.g., link orders.customer_id to customers.customer_id).
- **Sales Overview Report** - Create visuals for:
    1. Total sales over time (line or area chart)
    2. Monthly sales trend
    3. Total orders placed
- **Top Products by Sales** - Bar chart or table listing top-selling products by revenue or quantity.
- **Customer Purchase Analysis** - Add a stacked bar chart to show purchase patterns by city or state using the customers and orders table.
- **Sales by Store Map** - Use the map visual with store.state or store.zip_code to show sales distribution geographically.
- **Low Stock Alert Dashboard** - Use conditional formatting and cards to display products with stock levels below a threshold (e.g., 10 units).
- **Interactive Filters and Slicers** - Add slicers for:
    1. Order Status
    2. Product Category
    3. Store
- **Staff Performance Report** - Table or chart showing total sales/revenue per staff member based on handled orders.
- **Consolidated Dashboard Page –**
    1. Final report page with KPIs:
    2. Total Revenue
    3. Active Customers
    4. Average Order Value
    5. Total Orders
- **Import customer_segments Table** - Load the segmentation result (exported from Python) from SQL into Power BI.
- **Visualize Customer Segments** - Use Pie Chart / Donut / Tree Map to show the distribution of segments (e.g., Loyal, New, At-Risk).
- **Segment-Level Revenue Breakdown** - Add a bar chart that shows total revenue per customer segment.
- **Use Segments as Report Filters** - Enable filtering across dashboards by customer segment using slicers.

Technical Tags:

Excel, SQL, Python, Pandas, NumPy, Power BI, KMeans, EDA, ETL, Data Cleaning, Retail Analytics

Data Set Explanation:

- customers: Buyer information including contact and location
- orders: Transaction log with staff and store references
- order_items: Detailed item info per order
- products: Item details with category and brand
- brands/categories: Metadata about items
- stocks: Inventory across stores
- stores/staffs: Organizational structure

Project Deliverables:

- Cleaned Excel files
- SQL database creation scripts and query scripts
- Python notebook or script with analysis and ML
- Power BI .pbix file with visuals
- Final report documenting steps, insights, and visuals
- Project Guidelines:
- Use proper naming conventions and comments in SQL/Python
- Keep backups of Excel and SQL stages
- Use GitHub or local versioning (if collaborative)
- Follow visualization best practices (label axes, use tooltips, limit clutter)

Conclusion:

This project delivers a complete end-to-end retail analytics solution by leveraging Excel, SQL, Python, and Power BI to transform raw data into actionable insights. It encompasses the full data pipeline, including data cleaning, database management, analysis, and visualization, while utilizing SQL for efficient querying and Python for exploratory data analysis and RFM-based customer segmentation to gain deeper insights into customer behaviour and sales performance. The final Power BI dashboards present key business metrics in an interactive and visually compelling manner, supporting effective data-driven decision-making. Overall, the project demonstrates strong analytical and technical capabilities along with the ability to derive meaningful business insights in a real-world retail environment.